# EX07 - Taylor Approximation of exp(x) - Symbols and Lists

Beginner Step-by-Step Guide

This guide follows the real file `ex07_taylor_exp.py`. It explains the problem first and then the code from the perspective of someone starting Python.

## 1. What we want to solve

Build the Maclaurin polynomial of e^x term by term up to order N, evaluate it at xp and compare with exp(xp).

### Problem variables / elements

| Name | Meaning |
|---|---|
| N | highest order |
| x | algebraic symbol |
| xp | numeric evaluation point |
| ex | accumulated symbolic sum |

### Equation or logic on paper

```
e^x = sum_{n=0..N} x**n / n!
```

### Manual check

For N=2: 1+x+x**2/2. At x=0.5 it gives 1.625, while exp(0.5)~1.64872.

## 2. What we already know and what is new

**Already seen:**

- for
- range
- accumulators

**New in this exercise:**

- SymPy
- sp.symbols()
- empty list and append()
- join()
- subs()
- incremental factorial

# 3. New Python concepts

## 1. Symbol

**Syntax:** `x = sp.symbols("x")`

x is no longer a specific number; it becomes an algebraic symbol inside an expression.

## 2. List

**Syntax:** `terms = []`

[] creates an empty list that can grow with append().

## 3. append

**Syntax:** `terms.append(term)`

Adds term as one new element at the end of the list.

## 4. join

**Syntax:** `" + ".join(...)`

Joins a sequence of strings using " + " as the separator.

## 5. Substitution

**Syntax:** `ex_symbolic.subs(x, xp)`

Replaces symbol x with value xp in the expression.

# 4. Code step by step

## Step 1. Prepare variables

```
N = 5
x = sp.symbols("x")
xp = 0.5
ex = sp.Integer(0)
fact = 1
terms = []
```

ex is the running sum, fact is the current factorial and terms stores terms for display.

## Step 2. Iterate from n=0

```
for n in range(0, N + 1):
```

The series begins with n=0.

## Step 3. Create and store the term

```
term = x**n / fact
ex += term
terms.append(term)
```

Each iteration contributes x**n/n! to the polynomial.

## Step 4. Update factorial

```
fact *= n + 1
```

Doing this at the end keeps fact=1 for n=0 and prepares the next iteration.

## Step 5. Evaluate

```
ex_numeric = ex_symbolic.subs(x, xp)
```

After building the symbolic expression, x is replaced by 0.5.

## Step 6. Compare

```
analytical_result = exp(xp)
```

math.exp provides a numeric reference.

## 5. Execution flow

```
N,xp
 -> build symbolic terms 0..N
 -> polynomial ex
 -> substitute xp
 -> compare exp(xp)
```

## 6. Expected result / check

With N=5 and xp=0.5, the source obtains ~1.6486979167 versus 1.6487212707.

## 7. Common beginner mistakes

- Starting at n=1 and losing the term 1.
- Updating fact before using it.
- Confusing symbolic x with numeric xp.
- Using join on objects without converting them to str.

## 8. If you remember only five things

- SymPy supports symbolic algebra.
- append adds items to a list.
- The series is built term by term.
- Factorial can be updated incrementally.
- An approximation should be compared with a reference.

## 9. Mini-practice

Change one input. Before running, write a comment predicting what should change and why. Then run the file and compare your prediction with the output.

## Completion criterion

Move on when you can explain the problem, equation/logic, new Python concepts and execution flow without simply reading the code.